

01 **CONCEPTO & JERARQUÍA**

¿QUÉ ES UNA EXCEPCIÓN?

> Evento que **interrumpe el flujo normal** del programa

> Si no se controla, el programa **se detiene**

> En Java toda excepción es un **objeto**

```
int resultado = 10 / 0; // ArithmeticException
```

JERARQUÍA DE EXCEPCIONES

Throwable |— Error ← problemas graves del sistema |— OutOfMemoryError

← NO capturar |— Exception ← manejables |— RuntimeException ← Unchecked

|— NullPointerException |— ArithmeticException |—

ArrayIndexOutOfBoundsException |— IOException ← Checked |— SQLException

← Checked

CHECKED VS UNCHECKED

TIPO	OBLIGATORIO MANEJAR	EJEMPLOS
Checked	SÍ	IOException, SQLException
Unchecked	NO	NullPointerException, Arithmetic
Error	NO	OutOfMemoryError

EXCEPCIONES COMUNES

EXCEPCIÓN	CAUSA TÍPICA
NullPointerException	Usar objeto null
ArithmeticException	División por cero
ArrayIndexOutOfBoundsException	Índice fuera de rango
NumberFormatException	Parsear texto no numérico
IllegalArgumentException	Argumento inválido
IOException	Error de entrada/salida
InputMismatchException	Tipo incorrecto en Scanner

Regla: Error → nunca capturar. Checked → obligatorio manejar o declarar con throws. Unchecked → opcional pero recomendable.

02 **TRY - CATCH - FINALLY**

ESTRUCTURA BÁSICA TRY - CATCH

> El código **arriesgado** va dentro de **try**

> Cada **catch** captura un tipo específico de excepción

> El flujo **continúa** después del bloque catch

```
try {
    int resultado = 10 / 0;
} catch (ArithmeticException e) {
    System.out.println("División por cero");
}
// el programa continúa aquí
```

MÚLTIPLES CATCH - ORDEN IMPORTA

```
try {
    int[] nums = new int[3];
    nums[5] = 10; // índice inválido
} catch (ArrayIndexOutOfBoundsException e) {
    System.out.println("Índice fuera de rango");
} catch (Exception e) {
    System.out.println("Error general");
}
```

⚠ **Orden:** el **catch** más *específico* siempre antes que el general. Si pones **Exception** primero, los demás quedan inalcanzables.

FINALLY - SE EJECUTA SIEMPRE

> Se ejecuta **tanto si hay excepción como si no**

> Uso típico: **cerrar recursos**, conexiones, ficheros

```
try {
    int r = 10 / 2;
    System.out.println("Resultado: " + r);
} catch (ArithmeticException e) {
    System.out.println("Error matemático");
} finally {
    System.out.println("Siempre se ejecuta");
}
```

MENSAJE DE LA EXCEPCIÓN

```
try {
    int r = 10 / 0;
} catch (ArithmeticException e) {
    System.out.println(e.getMessage());
    e.printStackTrace(); // traza completa
}
```

MÉTODO	QUÉ DEVUELVE
getMessage()	Texto del error
printStackTrace()	Traza completa de llamadas
getClass().getName()	Nombre de la excepción

03 **THROW - THROWS - CUSTOM**

THROW - LANZAR EXCEPCIÓN MANUALMENTE

> Se usa **dentro** de un método

> Lanza una excepción en el momento deseado

```
public void retirar(double cantidad) {
    if (cantidad <= 0) {
        throw new IllegalArgumentException(
            "Cantidad inválida"
        );
    }
}
```

THROWS - DECLARAR EN LA FIRMA DEL MÉTODO

> Se usa en la **firma** del método (Checked)

> Obliga al **llamador** a manejarla o propagarla

```
public void leerArchivo() throws IOException {
    // código que puede lanzar IOException
}

// El llamador DEBE manejarla:
try {
    leerArchivo();
} catch (IOException e) {
    System.out.println("Error E/S");
}
```

THROW VS THROWS

	THROW	THROWS
Dónde	Cuerpo del método	Firma del método
Qué hace	Lanza la excepción	La declara/propaga
Cantidad	Una excepción	Varias con coma

EXCEPCIONES PERSONALIZADAS

> Extienden **Exception** (Checked) o **RuntimeException** (Unchecked)

> Adaptan los errores a la **lógica del negocio**

```
// Definición
public class SaldoInsuficienteException
    extends Exception {
    public SaldoInsuficienteException(String msg) {
        super(msg);
    }
}

// Uso
public void retirar(double cantidad)
    throws SaldoInsuficienteException {
    if (cantidad > saldo) {
        throw new SaldoInsuficienteException(
            "Saldo insuficiente");
    }
    saldo -= cantidad;
}
```

04 BUENAS PRÁCTICAS

QUÉ HACER Y QUÉ EVITAR

✓ SÍ

✓ Capturar solo lo necesario

✓ Usar mensajes descriptivos

✓ Crear excepciones propias cuando tenga sentido

✓ Cerrar recursos en finally

✓ Poner el catch más específico primero

✗ NO

✗ Usar catch(Exception) para todo

✗ Dejar catch vacíos sin mensaje

✗ Usar excepciones para controlar flujo normal sentido

✗ No lanzar excepción cuando corresponde

✗ Confundir throw y throws

EJEMPLO INTEGRADOR – CUENTABANCARIA

```
public class SaldoInsuficienteException
    extends Exception {
    public SaldoInsuficienteException(String m) {
        super(m);
    }
}

public class CuentaBancaria {
    private double saldo;

    public CuentaBancaria(double ini) {
        if (ini < 0)
            throw new IllegalArgumentException("Inválido");
        this.saldo = ini;
    }

    public void retirar(double c)
        throws SaldoInsuficienteException {
        if (c ≤ 0)
            throw new IllegalArgumentException("Cantidad inválida");
        if (c > saldo)
            throw new SaldoInsuficienteException("Saldo insuficiente");
        saldo -= c;
    }

    public double obtenerSaldo() { return saldo; }
}

public class Main {
    public static void main(String[] args) {
        try {
            CuentaBancaria c = new CuentaBancaria(1000);
            c.retirar(1500);
        } catch (SaldoInsuficienteException e) {
            System.out.println("Error: " + e.getMessage());
        } catch (IllegalArgumentException e) {
            System.out.println("Ang: " + e.getMessage());
        }
        System.out.println("Programa continúa...");
    }
}
```

Ej. EJERCICIOS 1 - 5

01 División por cero

- Programa que intente dividir dos números
- Capturar `ArithmeticException` con mensaje descriptivo
- El programa debe continuar tras el error

Objetivo: *sintaxis básica de try-catch*

02 IllegalArgumentException

- Método que recibe un número entero
- Si el número es negativo → lanzar `IllegalArgumentException`
- En `main` : llamar con valor negativo y capturar

Objetivo: *uso de throw manualmente*

03 EdadInvalidaException

- Crear clase `EdadInvalidaException` `extends Exception`
- Método que recibe una edad
- Si `edad < 0` o `> 120` → lanzar la excepción personalizada
- En `main` : capturar y mostrar el mensaje

Objetivo: *crear y usar excepción personalizada*

04 Validación de contraseña

- Método `validarContrasena(String c)`
- Si la contraseña tiene menos de 8 caracteres → lanzar excepción
- Crear `ContrasenaInvalidaException` propia
- Capturar en `main` con mensaje de error

Objetivo: *excepción personalizada + validación*

05 InputMismatchException

- Usar `Scanner` para leer un número por teclado
- Si el usuario escribe texto → capturar `InputMismatchException`
- Mostrar mensaje de error claro al usuario

Objetivo: *manejo de excepciones con entrada de usuario*

Ej. EJERCICIOS 6 - 9

06 Índice inválido

- Crear un array de 5 elementos
- Intentar acceder al índice 10
- Capturar `ArrayIndexOutOfBoundsException`
- Mostrar el índice que causó el error

Objetivo: *capturar excepción de acceso a array*

07 Propagación con throws

- Método `metodoB()` que lanza `Exception` con `throw`
- Método `metodoA()` que llama a `metodoB()` y declara `throws Exception`
- En `main` : llamar a `metodoA()` dentro de `try-catch`

Objetivo: *entender la propagación en la pila de llamadas*

08 try - catch - finally

- Bloque `try` que realiza una operación (p.ej. división)
- Bloque `catch` que muestra el error
- Bloque `finally` que imprime "Recursos liberados"
- Comprobar que `finally` se ejecuta siempre

Objetivo: *dominar los tres bloques juntos*

09 CuentaBancaria completa

- Clase `CuentaBancaria` con atributo `saldo`
- Crear `SaldoInsuficienteException` `extends Exception`
- Método `retirar()` que lanza excepción si `saldo < cantidad`
- En `main` : probar retiro normal y retiro imposible
- Capturar ambas excepciones con mensajes distintos

Objetivo: *integrar todo — excepción personalizada, throw, throws y try-catch*

RESUMEN DE CONCEPTOS CLAVE

try - catch

Encierra código peligroso y captura el error por tipo

finally

Siempre se ejecuta — ideal para cerrar recursos

throw

Lanza la excepción manualmente desde el cuerpo

throws

Declara en la firma que el método puede propagar